A Appendix

A.1 Proofs of Lemmas

Proof of Lemma 3.1: In the union operation, when we merge two window-CCs, we always add vertices in the smaller one into the larger one by updating the labels of the vertices in the smaller window-CC as the label of the larger one. Thus, for vertices in the smaller window-CC, the size of the window-CC containing them will be at least doubled after the union operation. Since the size of any window-CC is at most n, the label of each vertex is changed at most $(\log n)$ times.

Proof of Lemma 3.2: For each start time, the space cost is $O(n \log n)$ as there are n vertices and each vertex's label is changed $O(\log n)$ times. By enumerating all the t_{max} start times, we obtain Lemma 3.2.

Proof of Lemma 3.3: By Lemma 3.1, each $B(t_s, v)$ has at most $(\log n)$ $\langle label, time \rangle$ pairs. Thus, the binary search on each $B(t_s, v)$ costs $O(\log \log n)$ time, making the overall query time cost be $O(n \cdot \log \log n)$.

Proof of Lemma 3.4: There are $O(t_{max})$ start times and for each t_s, we examine at most m edges, and each vertex is involved in $O(\log n)$ union operations, so the overall time cost is $O((m + n \log n) t_{max})$.

Proof of Lemma 3.5: Suppose that $\Psi_{[t_s,t_s]}$ has been derived by an MST algorithm. We show that $\Psi_{[t_s,t_s+1]}$ can be derived based on $\Psi_{[t_s,t_s]}$. Specifically, we first initialize $\Psi_{[t_s,t_s+1]} = \Psi_{[t_s,t_s]}$, and then update it by the edges with timestamp $t_s + 1$. For each edge $(u, v, t_s + 1)$, if vertices u and v are not connected via the edges in $\Psi_{[t_s,t_s+1]}$, we add it into $\Psi_{[t_s,t_s+1]}$. After processing all the edges with $t = t_s + 1$, we obtain $\Psi_{[t_s,t_s+1]}$. By repeating the above process, we can derive a chain of TSFs satisfying the desired condition. Figure A.1 shows this process when we are constructing $\Psi_{[2,t]}$ of Figure 1.1(a).

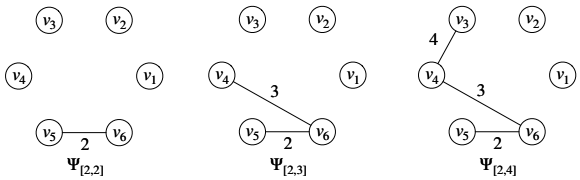

Fig. A.1: The nested relationship by anchoring $t_s = 2$.

Proof of Lemma 3.6: Suppose that $\Psi_{[0,t_{max}]}$ has been derived by the process in Lemma 3.5. We then can build $\Psi_{[1,t_{max}]}$ based on $\Psi_{[0,t_{max}]}$. Specifically, we first initialize $\Psi_{[1,t_{max}]}$ as $\Psi_{[0,t_{max}]} \cap E_{[1,t_{max}]}$, i.e. delete edges with timestamp 0 in $\Psi_{[0,t_{max}]}$, and then update it by the edges in $E_{[1,t_{max}]}$. Consider the edges in chronological order. For each edge (u, v, t), if vertices u and v are not connected via the edges in $\Psi_{[1,t_{max}]}$, we add it into $\Psi_{[1,t_{max}]}$. After processing all edges in $E_{[1,t_{max}]}$, we obtain $\Psi_{[1,t_{max}]}$. By repeating the above process, we can derive a set of TSFs $\Psi_{[0,t_{max}]}, \Psi_{[1,t_{max}]}, \cdots, \Psi_{[t_{max},t_{max}]}$. Since each edge with timestamp t will be added into Ψ for some $\widehat{t}$ and deleted in the initialization of $\Psi_{[t+1,t_{max}]}$, the condition in the lemma holds. Figure A.2 shows this process when we are constructing $\Psi_{[2,t_{max}]}$ from $\Psi_{[1,t_{max}]}$ of Figure 1.1(a).

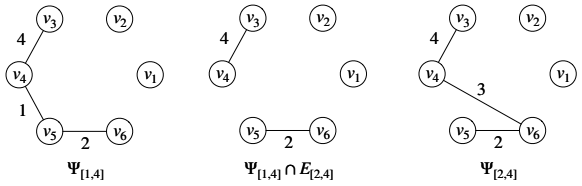

Fig. A.2: The overlapping relationship between $t_s = 1, 2$

Proof of Lemma 3.7: By Lemma 3.6, each edge has only one appearing time interval, so it appears exactly once in $F_p[\,]$ and once in $F_b[\,]$, making the overall space cost be $O(m)$.

Proof of Lemma 3.8: Note that the for loops in lines 4, 7, and 10 run $O(\sqrt{t_{max}})$ times. The accumulative time cost of the three inner loops equals $|E'| = O(n)$ (lines 5-6, 8-9, 11-12). Thus, the total time cost of fetching E' from the TSF-index is $O(n+\sqrt{t_{max}})$. Since the graph G' has n vertices and at most $n-1$ edges and it represents the TSF over $[t_s, t_e]$, i.e., $\Psi_{[t_s,t_e]}$, finding all the CCs from G' can be completed in $O(n)$ time (line 13). The total time cost of Algorithm 3 is $O(n + \sqrt{t_{max}})$.

Proof of Lemma 3.9: In Algorithm 4, each edge with timestamp t_j will be iterated for $O(t_j)$ times, since it would be considered for each $t_i \in [0, t_j]$. Hence, the total time cost is bounded by $O(m t_{max})$.

Proof of Lemma 3.10: We analyze the membership of the new edge $e_{new} = (u, v, t)$ in the TSF $\Psi_{[t_s,t]}$ with respect to the start time t_s:

1. **Case** $t_s \leq t_{min}$**:** The interval $[t_s, t]$ contains t_{min}. Consequently, u and v are already connected by $\mathcal{P}_{uv}$. e_{new} is not yet required to maintain connectivity.

2. **Case** $t_s > t_{min}$**:** The interval $[t_s, t]$ does not contain t_{min}. Thus, the edge e_{min} is invalid in this time window, and the path $\mathcal{P}_{uv}$ is broken. To maintain the connectivity of the MSF, e_{new} must be included to reconnect the component previously connected by e_{min}.

Therefore, e_{new} contributes to $\Psi_{[t_s,t]}$ if and only if the old path is broken, which occurs when $t_s > t_{min}$. Thus, the appearing time is $\widehat{t} = t_{min} + 1$.

Proof of Lemma 3.11: For each edge $(u, v, t) \in E$, Algorithm 5 inserts the edge into F in $O(1)$ time and updates Υ with (u, v, t) in $O(\tau)$ time. Therefore, the time complexity of Algorithm 5 is $O(m(1 + \tau)) = O(m\tau)$.

Proof of Lemma 3.12: The LCT maintains an MSF structure, which contains at most $n-1$ edges at any time. Therefore, the time complexity of maintaining the TSF-index is only related to n, and is $O(\log n)$ time by [58].

Proof of Lemma 4.1: For each start time t_s, the space cost is $O(n)$ as $|D(t_s)| \leq n$. By enumerating all the t_{max} start times, the lemma holds.

Proof of Lemma 4.2: Since $D(t_s)$ is a forest structure with n vertices, its number of edges is at most $n-1$. Hence, the total time cost is $O(n)$.

Proof of Lemma 4.3: For each time window $[t_s, t_e]$, the time complexity of finding all SCCs in $G_{[t_s,t_e]}$ is $O(m_{[t_s,t_e]} + n)$ [24, 56, 61], so the overall time complexity is $O\left(\sum_{t_s,t_e} m_{[t_s,t_e]} + n\right)$. Since $m_{[t_s,t_e]} \leq m$, the complexity is also bounded by $O((m+n)t_{max}^2)$.

Proof of Lemma 4.4: We prove the lemma by giving a method to construct such a chain. Suppose that $\Phi_{[t_s,t_s]}$ has been derived by invoking Algorithm 8 on each SCC of $G_{[t_s,t_s]}$. We show that $\Phi_{[t_s,t_s+1]}$ can be derived based on $\Phi_{[t_s,t_s]}$. We shrink each SCC of $G_{[t_s,t_s]}$ into a vertex and update $G_{[t_s,t_s+1]}$ with the shrinked vertices accordingly. For each SCC in the updated $G_{[t_s,t_s+1]}$, we use Algorithm 8 to find edges that strongly connect the SCC. By combining those edges with $\Phi_{[t_s,t_s]}$, we obtain $\Phi_{[t_s,t_s+1]}$. By repeating the above process, we can derive a chain of RES's satisfying the desired condition. Figure A.3 shows this process by constructing $\Phi_{[0,t_{max}]}$ for the graph in Figure 1.1(b).

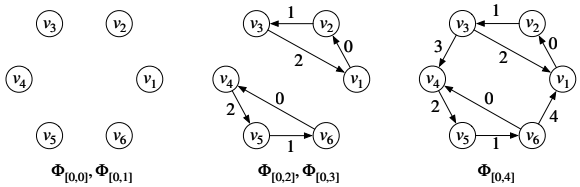

Fig. A.3: The nested relationship by anchoring $t_s = 0$

Proof of Lemma 4.5: We prove the lemma by contradiction. Assume $G'' = (V, E_1 \cup E_2)$ has a different set of SCCs as that of $G' = (V, E_1)$, then let $E_2 = E_{[t_s,t_e]}$, $G'' = G_{[t_s,t_e]}$ has inconsistent SCCs with $G' = (V, E_1)$. This contradicts Definition 4.1.

Proof of Lemma 4.6: Since $\Phi_{[t_s,t_e]} \subseteq \Phi_{[t_s,t_{max}]}$ and $\Phi_{[t_s,t_e]} \subseteq E_{[t_s,t_e]}$, there exists a valid RES satisfying $\Phi_{[t_s,t_e]} \subseteq \Phi_{[t_s,t_{max}]} \cap E_{[t_s,t_e]}$. By Lemma 4.5, $\Phi_{[t_s,t_{max}]} \cap E_{[t_s,t_e]}$ is also a valid RES.

Proof of Lemma 4.7: We prove the lemma by giving a method to construct a set of RES's satisfying the condition in the lemma. Suppose that $\Phi_{[0,t_{max}]}$ has been derived by the process in Lemma 4.4. We then can build $\Phi_{[1,t_{max}]}$ based on $\Phi_{[0,t_{max}]}$. Specifically, when calling Algorithm 8 to collect edges for $\Phi_{[1,t_{max}]}$, the edges in $\Phi_{[0,t_{max}]}$ have the privilege to be traversed first when performing BFS. By repeating the above process, we can derive a set of RES's $\Phi_{[0,t_{max}]}$, $\Phi_{[1,t_{max}]}, \cdots, \Phi_{[t_{max},t_{max}]}$. Since each edge with timestamp t will be added into Φ for some t_l and deleted for some $t_r + 1$, the condition in the lemma holds.

Proof of Lemma 4.8: Each edge has only one appearing time interval, so it appears exactly once in $R_P[\,]$ and once in $R_b[\,]$, making the overall space cost be $O(m)$.

Proof of Lemma 4.9: Similar to the proof of Lemma 3.8: since the constructed edge set E' corresponds to a valid $\Phi_{[t_s,t_e]}$ and $|\Phi_{[t_s,t_e]}| = O(n)$, the lines 2-12 in Algorithm 9 cost $O(n + \sqrt{t_{max}})$ time. The line 13 finishes in $O(n)$ time. Thus, the total time complexity of Algorithm 9 is $O(n + \sqrt{t_{max}})$.

Proof of Lemma 4.10: We prove the lemma by contradiction. Assume $T(t_s, e) < T(t_s - 1, e)$. Then $T(t_s, e) \leq t_{max}$ and e is involved in an SCC of $G_{[t_s,T(t_s,e)]}$. Therefore, it should also be involved in an SCC of $G_{[t_s-1,T(t_s,e)]}$, which contradicts our assumption.

Proof of Lemma 4.11: For the start time t_s, the number of different time intervals is $T(t_s, e) - T(t_s - 1, e) + 1$. Therefore, the total number of different time windows is:

$$\sum_{t_s=0}^{t_{max}} (T(t_s, e) - T(t_s - 1, e) + 1) = t_{max} + 1 + t_{max} = 2 \cdot t_{max} + 1,$$

where $T(-1, e) = 0$. The equation is asymptotic to $O(t_{max})$.

Proof of Lemma 4.12: By Lemma 4.11, each edge is processed $O(t_{max})$ times. Moreover, the number of edge processing steps dominates the time complexity of iterating over the vertices. Thus, Algorithm 10 achieves an overall time complexity of $O(mt_{max})$.

Proof of Lemma 5.1: For the cases $0 \leq t_s < t_i$ and $t < t_s \leq t_{\max}$, the claim follows immediately since e does not appear in $\Psi_{[t_s,t_{\max}]}$ for these time ranges.

For the case $t_i \leq t_s \leq t$, we proceed by contradiction. If no replacement edge e' is added, then u and v would not remain connected in $\Psi_{[t,t_{\max}]}$, contradicting the assumption that they stay in the same window-CC.

Since each TSF is a forest and there is an overlapping relationship between TSFs with adjacent t_s (as discussed in Section 3.2), at most one edge can be added as a replacement. If the added edge e' had a timestamp $< t$, this would contradict the TSF construction process: such an e' would have been inserted earlier than e, preventing e from ever appearing in $\Psi_{[t_s,t_{\max}]}$.

Therefore, a replacement edge e' with timestamp $\geq t$ must exist, completing the proof.

A.2 Interval-based TSF-index

We present the general TSF-index construction algorithm in Algorithm 11, which extends Algorithm 5 to support interval-based temporal edges. The core idea is to insert edges in non-decreasing order of their starting timestamps t_s (line 2). However, since each edge $(u, v, [t_s, t_e])$ remains active throughout the interval $[t_s, t_e]$, its representative timestamp in the MSF structure Υ is set to its ending time t_e (line 11). Similarly, the appearing interval of each edge ends at t_e; hence, the edge is inserted into the corresponding set $F(\cdot, t_e)$ (lines 8 and 10). When an edge connects two vertices already linked in Υ, the algorithm identifies the edge on the connecting path with the minimum timestamp $t_{\min}$ and replaces it if $t_{\min} < t_e$. Otherwise, the edge is skipped (lines 4–7). It is straightforward to verify the extended TSF-index has the same index construction time and space complexities as the original one.

It is worth noting that when all edges satisfy $t_s = t_e$, the graph degenerates to the discrete-time case considered in our basic problem definition, and Algorithm 11 becomes identical to Algorithm 5.

For query processing based on the extended index, although each $F(t_i, t_j)$ still represents the set of edges appearing in the TSFs $\Psi_{[t_i,t_{\max}]}, \Psi_{[t_i+1,t_{\max}]}, \ldots, \Psi_{[t_j,t_{\max}]}$ (i.e., the appearing interval defined as in Lemma 3.6), minor adaptations are required.

In the original TSF-index, when querying a time window $[t_s, t_e]$, we retrieve all edges in $F(t_i, t_j)$ where $0 \leq t_i \leq t_s$

Algorithm 12: TSF-query-extended($F_p[\,], F_b[\,], t_s, t_e$)

Input: the TSF-index $F_p[\,]$ and $F_b[\,]$, and query time window $[t_s, t_e]$;
Output: all the window-CCs in $G_{[t_s,t_e]}$;

1 $G' \leftarrow (V, E')$ where $E' = \emptyset$;
2 $l \leftarrow \lfloor \frac{t_s}{\sqrt{t_{max}}} \rfloor + 1$;
3 $r \leftarrow \lfloor \sqrt{t_{max}} \rfloor$;
4 **for** $t_j \in [t_s, l\sqrt{t_{max}})$ **do**
5 **for** $F(t_i, t_j) \in F_p[t_j] \wedge t_i \in [0, t_s]$ **do**
6 $E' \leftarrow E' \cup F(t_i, t_j)$;
7 **for** $x \in [l, r)$ **do**
8 **for** $F(t_i, t_j) \in F_b[x] \wedge t_i \in [0, t_s]$ **do**
9 $E' \leftarrow E' \cup F(t_i, t_j)$;
10 **for** $t_j \in [r\sqrt{t_{max}}, t_{max}]$ **do**
11 **for** $F(t_i, t_j) \in F_p[t_j] \wedge t_i \in [0, t_s]$ **do**
12 $E' \leftarrow E' \cup F(t_i, t_j)$;
13 Filter edges in E' to keep edges with time interval intersecting $[t_s, t_e]$;
14 **return** all the CCs computed from $G' = (V, E')$;

Algorithm 13: RES-construct-extended(G)

Input: a directed temporal graph $G = (V, E)$;
Output: the corresponding RES-index;

1 $E_t \leftarrow$ edges starting at t;
2 **for** $t_i \leftarrow 0, \cdots, t_{max}$ **do**
3 $G' \leftarrow (V', E')$ where $V' \leftarrow \emptyset, E' \leftarrow \emptyset$;
4 **for** $t_j \leftarrow t_i, \cdots, t_{max}$ **do**
5 $\Phi_{[t_i,t_j]} \leftarrow \emptyset$;
6 **if** $t_j > t_i$ **then** $\Phi_{[t_i,t_j]} \leftarrow \Phi_{[t_i,t_j-1]}$;
7 **for** $e = (u, v, [t_s, t_e]) \in E_{t_j}$ **do**
8 Map u, v to u', v';
9 $E' \leftarrow E' \cup \{e' = (u', v')\}$;
10 $V' \leftarrow V' \cup \{u', v'\}$;
11 Clear E_{t_j} and run algorithms to find SCCs on G';
12 **for** *each SCC in* G' **do**
13 **for** *each internal edge* e' *in the SCC* **do**
14 $e \leftarrow$ the original edge of e'; Add e into E_{t_j};
15 Find RES edges E_S for the SCC by calling Algorithm 8;
16 Add E_S to $\Phi_{[t_i,t_j]}$ and shrink the SCC into a vertex;
17 **for** $e \in \Phi_{[t_i,t_{max}]} \backslash \Phi_{[t_i-1,t_{max}]}$ **do** $t_l(e) \leftarrow t_i$;
18 **for** $e \in \Phi_{[t_i-1,t_{max}]} \backslash \Phi_{[t_i,t_{max}]}$ **do** add e into $R(t_l(e), t_i - 1)$;
19 **if** $t_i \neq t_{max}$ **then**
20 **foreach** $e = (u, v, [t_s, t_e]) \in E_{t_i}$ **do**
21 **if** $t_e \geq t_i + 1$ **then** $E_{t_i+1} \leftarrow E_{t_i+1} \cup \{e\}$;
22 **for** $t_i \leftarrow 0, \cdots, t_{max}$ **do**
23 **for** $t_j \leftarrow t_i, t_i + 1, \cdots, t_{max} \wedge R(t_i, t_j) \neq \emptyset$ **do**
24 append $R(t_i, t_j)$ to $R_p[t_j]$ and $R_b[\lfloor \frac{t_j}{\sqrt{t_{max}}} \rfloor]$;

Algorithm 11: MSF-construct-extended(G)

Input: an undirected temporal $G = (V, E)$;
Output: the corresponding TSF-index;

1 $\Upsilon, F \leftarrow \emptyset$;
2 **foreach** $(u, v, [t_s, t_e]) \in E$ *in chronological order of* t_s **do**
3 **if** u *and* v *are connected in* Υ **then**
4 $P \leftarrow$ the path connecting u, v in Υ;
5 $(x, y, t_{min}) \leftarrow$ the edge with the minimum timestamp in P;
6 **if** $t_{min} \geq t_e$ **then continue**;
7 $\Upsilon \leftarrow \Upsilon \setminus \{(x, y, t_{min})\}$;
8 $F(t_{min} + 1, t_e) \leftarrow F(t_{min} + 1, t_e) \cup \{(u, v, [t_s, t_e])\}$;
9 **else**
10 $F(0, t_e) \leftarrow F(0, t_e) \cup \{(u, v, [t_s, t_e])\}$;
11 $\Upsilon \leftarrow \Upsilon \cup \{(u, v, t_e)\}$;
12 **for** $t_i \leftarrow 0, \cdots, t_{max}$ **do**
13 **for** $t_j \leftarrow t_i, t_i + 1, \cdots, t_{max} \wedge F(t_i, t_j) \neq \emptyset$ **do**
14 append $F(t_i, t_j)$ to $F_p[t_j]$ and $F_b[\lfloor \frac{t_j}{\sqrt{t_{max}}} \rfloor]$;

and $t_s \leq t_j \leq t_e$, which collectively correspond to the TSF $\Psi_{[t_s,t_e]}$. This works because, in the discrete-time model, the appearing interval of an edge with timestamp t also terminates at t, meaning that $F(t_i, t_j)$ contains edges with timestamp t_j. Thus, edges with $t_j > t_e$ are not yet active within the query window $[t_s, t_e]$.

However, in the extended TSF-index, each $F(t_i, t_j)$ may contain edges with active intervals $[t, t_j]$ for some $t \leq t_j$, implying that some of these edges may already be active before t_j. To correctly capture all edges active within $[t_s, t_e]$, we first construct $\Psi_{[t_s,t_{\max}]}$ by retrieving all edges in $F(t_i, t_j)$ where $0 \leq t_i \leq t_s$ and $t_s \leq t_j \leq t_{\max}$. We then filter these edges to retain only those whose active intervals intersect with the query window $[t_s, t_e]$.

Since the size of $\Psi_{[t_s,t_{\max}]}$ remains bounded by $O(n)$, the overall query time complexity remains asymptotically unchanged. Algorithm 12 details the extended query procedure, which generalizes Algorithm 3. The main modifications are: (i) setting $r = \lfloor \sqrt{t_{\max}} \rfloor$, (ii) iterating t_j from t_s to $t_{\max}$ (lines 3 and 10), and (iii) filtering edges in line 13 whose active intervals intersect with the query window.

A.3 Interval-based RES-index

Before extending the RES-index to handle interval-based temporal edges, we first make an important observation. If an edge $e = (u, v, [t_s, t_e])$ cannot form any SCC within the interval $[t_s, t_{\max}]$, then it is equivalent to a discrete-time edge $e = (u, v, t_s)$. This is because such an edge will never participate in any SCC in $[t, t_{\max}]$ for all $t \geq t_s$, and therefore will not contribute to any RES in $\Phi_{[t,t_{\max}]}$. Based on this observation, these edges can be safely handled by the original algorithm without modification, and we only need to focus on those edges that can potentially form SCCs beyond their appearing time.

Algorithm 13 presents the general RES-index construction procedure that extends Algorithm 10 to support temporal edges with time intervals. The core idea is to first group edges by their starting timestamps (line 1). During the construction, edges that may continue contributing to SCCs after

the current time t_i are propagated to the next edge set E_{t_i+1} (lines 19–21). Edges that cannot form SCCs, as identified in the above observation, are naturally filtered out in line 11, thus avoiding unnecessary processing. Each edge is still processed at most $O(t_{\max})$ times, and the proof follows a similar manner as in Lemma 4.11, so we omit the details for brevity. In this way, the algorithm effectively supports interval-based edges while maintaining the same asymptotic complexity as the original RES-index construction.